Shaheed Zulfikar Ali Bhutto Institute of Science and Technology

Department of Computer Science

Operating System
Project Report
CPU Scheduling Visualizer

Submitted To:
Sir Khalil-ur-Rehman
Sir Wali Muhammad Khubaib

Submitted By:
Varsha Bai-2312389
Vishaka Shankar Golani-2312390

Date of Submission: [2-jan-2026]

Abstract

This project implements a CPU Scheduling Visualizer using C language to demonstrate core operating system scheduling concepts. The system allows users to input processes and visualizes their execution using Gantt charts. It supports FCFS, SJF, Priority, and Round Robin scheduling algorithms. Inter-process communication is achieved using pipes, while multi-threading is used to display the Gantt chart. A deadlock demonstration module shows how improper resource locking can cause deadlocks.

1. Introduction

CPU scheduling is a fundamental responsibility of an operating system. It determines how processes are allocated CPU time to ensure efficiency and fairness. This project simulates multiple scheduling algorithms and visually represents execution to improve understanding of OS behavior.

2. Literature Review

Operating systems use scheduling algorithms to optimize CPU utilization and response time. FCFS is simple but inefficient, while SJF minimizes waiting time. Priority scheduling assigns importance-based execution, and Round Robin ensures fairness using time quantum. Deadlocks occur when processes wait indefinitely for resources.

3. System Design and Implementation

3.1 Process Creation
User inputs processes which are transferred using pipes between parent and child processes.

3.2 Scheduling Algorithms
The system supports FCFS, SJF, Priority, and Round Robin scheduling.

3.3 Multi-threading
A separate thread is used to draw the Gantt chart after scheduling.

3.4 Inter-Process Communication
Pipes are used for communication between processes.

3.5 Deadlock Demonstration
Mutex locks simulate deadlock scenarios using two competing threads.

4. Applications and Use Cases

CPU scheduling is used in real-time systems, multitasking operating systems, cloud computing, and embedded systems.

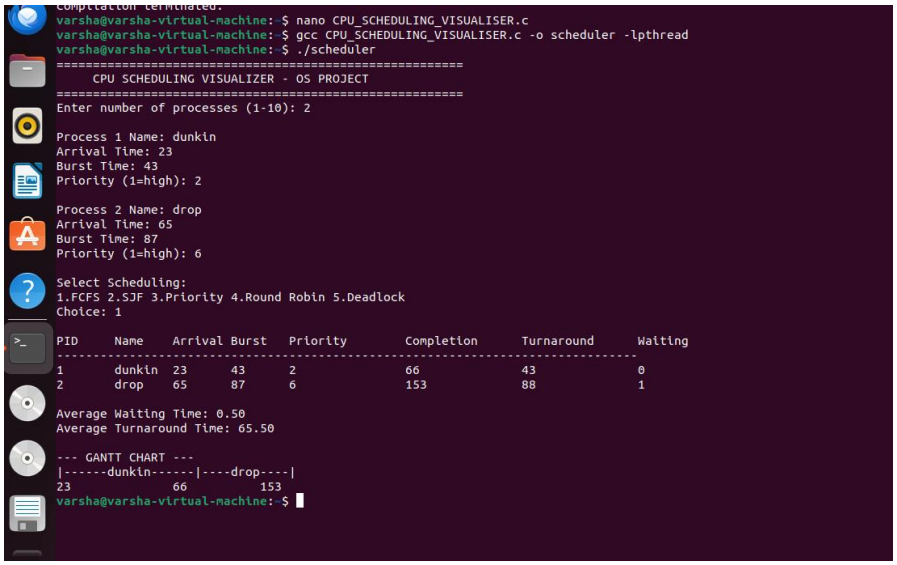
5. Challenges and Future Improvements

Challenges:
- Deadlock handling
- Accurate visualization
- Scheduling fairness

Future Improvements:
- GUI-based visualization
- Deadlock prevention algorithms
- Preemptive SJF

6. Output :

Choice 1:



```
varsha@varsha-virtual-machine:~$ nano CPU_SCHEDULING_VISUALISER.c
varsha@varsha-virtual-machine:~$ gcc CPU_SCHEDULING_VISUALISER.c -o scheduler -lpthread
varsha@varsha-virtual-machine:~$ ./scheduler
=========================================================
     CPU SCHEDULING VISUALIZER - OS PROJECT
=========================================================
Enter number of processes (1-10): 2

Process 1 Name: dunkin
Arrival Time: 23
Burst Time: 43
Priority (1=high): 2

Process 2 Name: drop
Arrival Time: 65
Burst Time: 87
Priority (1=high): 6

Select Scheduling:
1.FCFS 2.SJF 3.Priority 4.Round Robin 5.Deadlock
Choice: 1

PID     Name    Arrival Burst   Priority        Completion      Turnaround      Waiting
-----------------------------------------------------------------------------------
1       dunkin  23      43      2               66              43              0
2       drop    65      87      6               153             88              1

Average Waiting Time: 0.50
Average Turnaround Time: 65.50

--- GANTT CHART ---
|------dunkin------|----drop----|
23              66          153
varsha@varsha-virtual-machine:~$
```

Choice 2:

```
|------dunkin------|----drop----|
23                66          153
varsha@varsha-virtual-machine:~$ ./scheduler
=========================================================
     CPU SCHEDULING VISUALIZER - OS PROJECT
=========================================================
Enter number of processes (1-10): 1

Process 1 Name: cbtl
Arrival Time: 34
Burst Time: 85
Priority (1=high): 4

Select Scheduling:
1.FCFS 2.SJF 3.Priority 4.Round Robin 5.Deadlock
Choice: 2

PID     Name    Arrival Burst   Priority        Completion      Turnaround      Waiting
-------------------------------------------------------------------------------------
1       cbtl    34      85      4               119             85              0

Average Waiting Time: 0.00
Average Turnaround Time: 85.00

--- GANTT CHART ---
|----cbtl----|
34          119
```

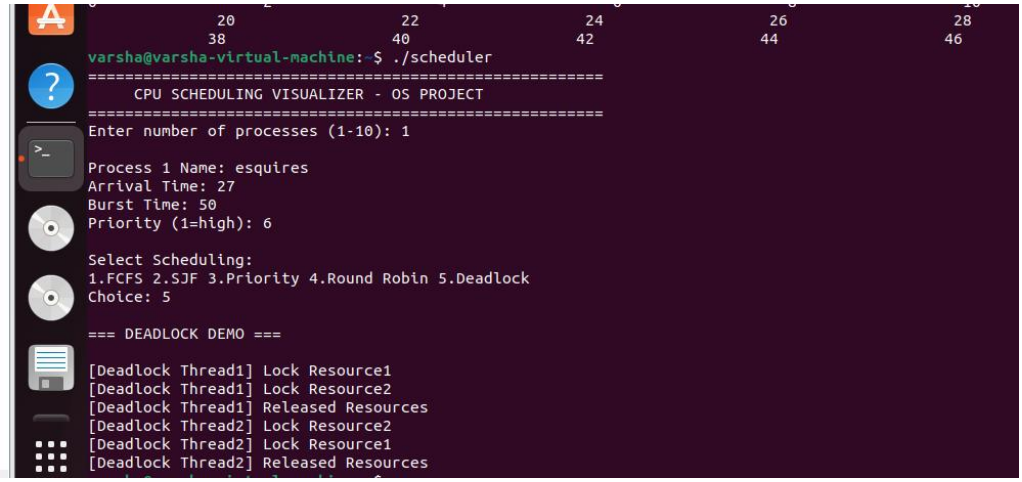
Choice 3:

```
|----cbtl----|
34          119
varsha@varsha-virtual-machine:~$ ./scheduler
=========================================================
     CPU SCHEDULING VISUALIZER - OS PROJECT
=========================================================
Enter number of processes (1-10): 1

Process 1 Name: senzo
Arrival Time: 21
Burst Time: 49
Priority (1=high): 3

Select Scheduling:
1.FCFS 2.SJF 3.Priority 4.Round Robin 5.Deadlock
Choice: 3

PID     Name    Arrival Burst   Priority        Completion      Turnaround      Waiting
-------------------------------------------------------------------------------------
1       senzo   21      49      3               70              49              0

Average Waiting Time: 0.00
Average Turnaround Time: 49.00

--- GANTT CHART ---
|-----senzo-----|
21            70
```

Choice 4:

```
21            70
varsha@varsha-virtual-machine:~$ ./scheduler
=========================================================
     CPU SCHEDULING VISUALIZER - OS PROJECT
=========================================================
Enter number of processes (1-10): 1

Process 1 Name: espresso
Arrival Time: 98
Burst Time: 69
Priority (1=high): 7

Select Scheduling:
1.FCFS 2.SJF 3.Priority 4.Round Robin 5.Deadlock
Choice: 4

PID     Name    Arrival Burst   Priority        Completion      Turnaround      Waiting
-------------------------------------------------------------------------------------
0               0       0       0               64              64              64

Average Waiting Time: 64.00
Average Turnaround Time: 64.00

--- GANTT CHART ---
|--------espresso--------|--------espresso--------|--------espresso--------
```

Choice 5:



7. Conclusion

The CPU Scheduling Visualizer successfully demonstrates OS scheduling algorithms, IPC, multi-threading, and deadlock concepts. It serves as an educational tool for understanding process execution and resource management.

References

1. Silberschatz et al., Operating System Concepts
2. Tanenbaum, Modern Operating Systems
3. GeeksforGeeks OS Tutorials